

Core Issue [1776](#): Replacement of class objects containing reference members

ISO/IEC JTC1 SC22 WG21
P0137R1
Richard Smith
richard@metafoo.co.uk
2016-06-23

And also:

Core Issue [1116](#): Aliasing of union members

Changes since [R0](#)

- creating a new object that exactly displaces a subobject of an existing object creates a new subobject of that existing object
- assigning to a union member access expression can change the active union member
- an array of `unsigned char` can be used to hold other objects
- reallo casts between pointer types to work in a small handful of C-compatible cases (between a struct and its first member etc)

Core wording

☐ Hide deleted text

Change in 1.8 (intro.object) paragraph 1:

The constructs in a C++ program create, destroy, refer to, access, and manipulate objects. **An object is created by a definition (3.1), by a new-expression (5.3.4), when implicitly changing the active member of a union ([class.union]), or when a temporary object is created ([conv.rval], [class.temporary]). An object occupies a region of storage in its period of construction (12.7 class.ctor), throughout its lifetime (3.8 basic.life), and in its period of destruction (12.7 class.dtor).** [Note: A function is not an object, regardless of whether or not it occupies storage in the way that objects do. — end note] **An object is created by a definition (3.1), by a new-expression (5.3.4) or by the implementation (12.2) when needed. [...]**

Drafting note: this maintains the status quo that `malloc` alone is not sufficient to create an object. Drafting note: the terms "period of construction" and "period of destruction" are in core issue 1517, which is not yet approved, but their use here seems sufficiently clear with or without that change.

Change in 1.8 (intro.object) paragraph 2:

Objects can contain other objects, called *subobjects*. A subobject can be a *member subobject* (9.2), a *base class subobject* (Clause 10), or an array element. An object that is not a subobject of any other object is called a *complete object*. **If an object is created in storage associated with a member subobject or array element *e* (which may or may not be within its lifetime), the created object is a subobject of *e*'s containing object if:**

- the lifetime of *e*'s containing object has begun and not ended, and
- the storage for the new object exactly overlays the storage location associated with *e*, and
- the new object is of the same type as *e* (ignoring cv-qualification).

[Note: If the subobject contains a reference member or a const subobject, the name of the original subobject cannot be used to access the new object ([basic.life]). — end note] [Example:

```
struct X { const int n; };
union U { X x; float f; };
void tong() {
    U u = {{ 1 }};
    u.f = 5.f; // OK, creates new subobject of 'u' (9.5)
    X *p = new (&u.x) X {2}; // OK, creates new subobject of 'u'
    assert(p->n == 2); // OK
    assert(*std::launder(&u.x.n) == 2); // OK
    assert(u.x.n == 2); // undefined behavior, 'u.x' does not name new subobject
}
```

— end example]

Add a new paragraph after 1.8 (intro.object) paragraph 2:

If a complete object is created (5.3.4) in storage associated with another object *e* of type "array of *N* unsigned char", that array provides storage for the created object if:

- the lifetime of *e* has begun and not ended, and
- the storage for the new object fits entirely within *e*, and
- there is no smaller array object that satisfies these constraints.

[*Note*: If that portion of the array previously provided storage for another object, the lifetime of that object ends because its storage was reused ([basic.life]). — *end note*] [*Example*:

```
template<typename ...T>
struct AlignedUnion {
    alignas(T...) unsigned char data[max(sizeof(T)...)];
};
int f() {
    AlignedUnion<int, char> au;
    int *p = new (au.data) int; // OK, au.data provides storage
    char *c = new (au.data) char(); // OK, ends lifetime of *p
    char *d = new (au.data + 1) char();
    return *c + *d; // OK
}
```

— *end example*]

Add another new paragraph after 1.8 (intro.object) paragraph 2:

An object *a* is nested within another object *b* if:

- *a* is a subobject of *b*, or
- *b* provides storage for *a*, or
- there exists an object *c* where *a* is nested within *c*, and *c* is nested within *b*.

Change in 1.8 (intro.object) paragraph 6:

Unless an object is a bit-field or a base class subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects **a and b with overlapping lifetimes** that are not bit-fields may have the same address if one is a subobject of **nested within** the other, or if at least one is a base class subobject of zero size and they are of different types; otherwise, they **shall** have distinct addresses. [Footnote: ...] [Example: ...]

Add a new paragraph after 3.7 (basic.stc) paragraph 3:

When the end of the duration of a region of storage is reached, the values of all pointers representing the address of any part of that region of storage become invalid pointer values ([basic.compound]). Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. Any other use of an invalid pointer value has implementation-defined behavior. [Footnote: Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault. — *end footnote*]

Drafting note: this should apply to all storage durations that can end, not just to dynamic storage duration. On an implementation supporting threads or segmented stacks, thread and automatic storage may behave in the same way that dynamic storage does.

Change in 3.7.4.1 (basic.stc.dynamic.allocation) paragraph 2:

[...] Furthermore, for the library allocation functions in 18.6.1.1 and 18.6.1.2, *p0* shall **point to represent the address of** a block of storage disjoint from the storage for any other object accessible to the caller. The effect of indirecting through a pointer returned as a request for zero size is undefined.

Change in 3.7.4.1 (basic.stc.dynamic.allocation) paragraph 2:

If the argument given to a deallocation function in the standard library is a pointer that is not the null pointer value (4.10), the deallocation function shall deallocate the storage referenced by the pointer, **ending the duration of the region of storage.** rendering invalid all pointers referring to any part of the deallocated storage. Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. Any other use of an invalid pointer value has implementation-defined behavior. [Footnote: Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault. — *end footnote*]

Change in 3.8 (basic.life) paragraph 1:

The *lifetime* of an object is a runtime property of the object. An object is said to have *non-vacuous initialization* if it

is of a class or aggregate type and it or one of its members is initialized by a constructor other than a trivial default constructor. [*Note*: initialization by a trivial copy/move constructor is non-vacuous initialization. — *end note*] The lifetime of an object of type τ begins when:

- storage with the proper alignment and size for type T is obtained, and
- if the object has non-vacuous initialization, its initialization is complete;

except that if the object is a union member or subobject thereof, its lifetime only begins if that union member is the initialized member in the union (8.5.1, 12.6.2), or as described in 9.5.

The lifetime of an object o of type τ ends when:

- if T is a class type with a non-trivial destructor (12.4), the destructor call starts, or
- the storage which the object occupies is **released, or is reused by an object that is not nested within o ([intro.object]) or released.**

Change in 3.8 (basic.life) paragraph 5:

Before the lifetime of an object has started but after the storage which the object will occupy has been allocated [Footnote: ...] or, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any pointer that **refers to** **represents the address of** the storage location where the object will be or was located may be used but only in limited ways. [...]

Change in 3.8 (basic.life) paragraph 6:

[...] The program has undefined behavior if:

- ~~an lvalue-to-rvalue conversion (4.1) is applied to such a glvalue~~ **the glvalue is used to access the object, or**
- the glvalue is used to ~~access a non-static data member or~~ call a non-static member function of the object, or

Change in 3.8 (basic.life) paragraph 7:

If, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, a new object is created at the storage location which the original object occupied, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object, if:

- [...]

[*Example*: ... — *end example*] **[*Note*: If these conditions are not met, a pointer to the new object can be obtained from a pointer that represents the address of its storage by calling `std::launder` (18.6 [support.dynamic]). — *end note*]**

Change in 3.8 (basic.life) paragraph 9:

Creating a new object ~~at the storage location~~ **within the storage** that a `const` **complete** object with static, thread, or automatic storage duration occupies, or, ~~at the storage location~~ **within the storage** that such a `const` object used to occupy before its lifetime ended, results in undefined behavior. [Example: ...]

This is necessary to allow types such as `std::optional` to contain `const` subobjects; the existing restriction exists to allow ROMability, and so only affects complete objects.

Change in 3.9.2 (basic.compound) paragraph 3:

[...] **Every value of pointer type is one of the following:**

- **a pointer to an object or function (the pointer is said to *point to* the object or function), or**
- **a pointer past the end of an object (5.7 `expr.add`), or**
- **the null pointer value (4.10 `conv.ptr`) for that type, or**
- **an invalid pointer value.**

A **valid** value of an object pointer type **that is a pointer to or past the end of an object** ~~represents either the address~~ **represents the address of a** ~~the first~~ byte in memory (1.7 `intro.memory`) **occupied by the object** [*Footnote*: For an object that is not within its lifetime, this is the first byte in memory that it will occupy or used to occupy.] **or the first byte in memory after the end of the storage occupied by the object, respectively**

or a null pointer (4.10). If an object of type T is located at an address A , a pointer of type $\text{cv } T^*$ whose value is the address A is said to point to that object, regardless of how the value was obtained. [Note: For instance, the address one **A pointer** past the end of an **array object** (5.7) ~~would be~~ **is not** considered to point to an unrelated object of the array's element **object's** type that might be located at that address. **There are further restrictions on pointers to objects with dynamic storage duration; see 3.7.4.3. A pointer value becomes invalid when the storage it denotes reaches the end of its storage duration; see [basic.stc].** — end note] For purposes of pointer arithmetic (5.7 **expr.add**) and comparison (5.9 **expr.rel**, 5.10 **expr.eq**), a pointer past the end of the last element of an array x **of n elements is considered to be equivalent to a pointer to a hypothetical element $x[n]$.** The value representation of pointer types is implementation-defined. Pointers to cv-qualified and cv-unqualified versions (3.9.3) of layout-compatible types shall have the same value representation and alignment requirements (3.11). [Note: ... — end note]

Add a new paragraph after 3.9.2 (basic.compound) paragraph 3:

Two objects a and b are *pointer-interconvertible* if:

- they are the same object, or
- one is a standard-layout union object and the other is a non-static data member of that object (9.5), or
- one is a standard-layout class object and the other is the first non-static data member of that object, or, if the object has no non-static data members, the first base class subobject of that object (9.2), or
- there exists an object c such that a and c are pointer-interconvertible, and c and b are pointer-interconvertible.

If two objects are pointer-interconvertible, then they have the same address, and it is possible to obtain a pointer to one from a pointer to the other via a `reinterpret_cast` (`[expr.reinterpret.cast]`). [Note: An array object and its first element are not pointer-interconvertible, even though they have the same address. — end note]

Change in 4.10 (conv.ptr) paragraph 2:

A prvalue of type "pointer to cv T ", where T is an object type, can be converted to a prvalue of type "pointer to cv void". ~~The result of converting a non-null pointer value of a pointer to object type to a "pointer to cv void" represents the address of the same byte in memory as the original pointer value. The null pointer value is converted to the null pointer value of the destination type.~~ **The pointer value (3.9.2) is unchanged by this conversion.**

Change in 5.2.9 (expr.static.cast) paragraph 13:

A prvalue of type "pointer to cv1 void" can be converted to a prvalue of type "pointer to cv2 T ", where T is an object type and cv2 is the same cv-qualification as, or greater cv-qualification than, cv1. ~~The null pointer value is converted to the null pointer value of the destination type.~~ If the original pointer value represents the address A of a byte in memory and A ~~satisfies~~ **does not satisfy** the alignment requirement of T , then the resulting pointer value **is unspecified** ~~represents the same address as the original pointer value, that is, A .~~ **Otherwise, if the original pointer value points to an object a , and there is an object b of type τ (ignoring cv-qualification) that is pointer-interconvertible (3.9.2) with a , the result is a pointer to b . Otherwise, the pointer value is unchanged by the conversion.** The result of any other such pointer conversion is unspecified. A value of type pointer to object converted to "pointer to cv void" and back, possibly with different cv-qualification, shall have its original value.

Change in 5.7 (expr.add) paragraph 4:

When an expression that has integral type is added to or subtracted from a pointer, the result has the type of the pointer operand. ~~If the pointer operand points to an element of an array object [Footnote], and the array is large enough, the result points to an element offset from the original element such that the difference of the subscripts of the resulting and original array elements equals the integral expression. In other words, if~~ **If the expression P points to the i -th element $x[i]$ of an array object x with n elements [Footnote: An object that is not an array element is considered to belong to a single-element array for this purpose; see 5.3.1. A pointer past the end of the last element of an array x of n elements is considered to be equivalent to a pointer to a hypothetical element $x[n]$ for this purpose; see 3.9.2.], the expressions $(P)+N$ (equivalently, $N+(P)$) and $(P)-N$ $P + J$ and $J + P$ (where NJ has the value nj) point to, respectively, the (possibly hypothetical) element $x[i + j]$ -th and $x[i - j]$ -th elements of the array object, provided they exist **if $0 \leq i + j \leq n$** Moreover, if the expression P points to the last element of an array object, the expression $(P)+1$ points one past the last element of the array object, and if the expression Q points one past the last element of an array object, the expression $(Q)-1$ points to the last element of the array object. If both the pointer operand and the result point to elements of the same array object, or one past the last element of the array object, the evaluation shall not produce an overflow; otherwise, the behavior is undefined. **Likewise, the expression $P - J$ points to the (possibly hypothetical) element $x[i - j]$ if $0 \leq i - j \leq n$; otherwise, the behavior is undefined.****

Change in 5.7 (expr.add) paragraph 5:

When two pointers to elements of the same array object are subtracted, the result is the difference of the subscripts of the two array elements. The type of the result is an implementation-defined signed integral type; this type shall be the same type that is defined as `std::ptrdiff_t` in the `<cstdlib>` header (18.2). As with any other arithmetic overflow, if the result does not fit in the space provided, the behavior is undefined. In other words, if the expressions `P` and `Q` point to, respectively, the i -th and j -th elements `x[i]` and `x[j]` of an array object `x`, the expression `(P)-(Q)` has the value $i-j$ provided the value fits in an object of type `std::ptrdiff_t`; otherwise, the behavior is undefined. [**Note: If the value $i-j$ is not in the range of representable values of type `std::ptrdiff_t`, the behavior is undefined. — end note**] Moreover, if the expression `P` points either to an element of an array object or one past the last element of an array object, and the expression `Q` points to the last element of the same array object, the expression `((Q)+1)-(P)` has the same value as `((Q)-(P))+1` and as `-(P)-((Q)+1)`, and has the value zero if the expression `P` points one past the last element of the array object, even though the expression `(Q)+1` does not point to an element of the array object. Unless both pointers point to elements of the same array object, or one past the last element of the array object, the behavior is undefined. [Footnote: Another way to approach pointer arithmetic ...]

Change in 5.7 (expr.add) paragraph 7:

If the value 0 is added to or subtracted from a **null** pointer value, the result compares equal to the original pointer value **is a null pointer value**. If two pointers point to the same object or both point past the end of the same array or both are null, and the two pointer values are subtracted, the result compares equal to the value 0 converted to the type `std::ptrdiff_t`.

Change in 5.9 (expr.rel) paragraph 3:

Comparing **unequal** pointers to objects [*Footnote: An object that is not an array element is considered to belong to a single-element array for this purpose; see 5.3.1. A pointer past the end of the last element of an array of n elements is considered to be equivalent to a pointer to a hypothetical element n for this purpose; see 3.9.2.] is defined as follows:*

- If two pointers point to different elements of the same array, or to subobjects thereof, the pointer to the element with the higher subscript compares greater.
- ~~If one pointer points to an element of an array, or to a subobject thereof, and another pointer points one past the last element of the array, the latter pointer compares greater.~~
- If two pointers point to different non-static data members of the same object, or to subobjects of such members, recursively, the pointer to the later declared member compares greater provided the two members have the same access control (Clause 11) and provided their class is not a union.

Drafting note: the change in 3.9.2 affects the semantics of an example like:

```
struct S {
    char a;
    int b;
} s;
static_assert(static_cast<void*>(&s.b) >= static_cast<void*>(&s.a + 1));
```

*Prior to this change, the result of the comparison was unspecified; now, it is required to hold, because `&s.a + 1` is treated as a pointer to an element of the "array" `s.a`. The value of `&s.b > &s.a + 1` is now specified and is *false* if and only if the pointers represent the same address.*

Change in 5.20 expr.const paragraph 2 bullet 8:

an lvalue-to-rvalue conversion (4.1) ~~or modification (5.18, 5.2.6, 5.3.2)~~ that is applied to a glvalue that refers to a non-active member of a union or a subobject thereof;

Add a new bullet after 5.20 expr.const paragraph 2 bullet 8:

an assignment expression (5.18) or invocation of an assignment operator (12.8) that would change the active member of a union;

Change in 9.2 class.mem paragraph 19:

In a standard-layout union with an active member (9.5) of struct type `T1`, it is permitted to read a non-static data member `m` of another union member of struct type `T2` provided `m` is part of the common initial sequence of `T1` and `T2`; **the behavior is as if the corresponding member of `T1` were nominated.** [**Example:**


```

struct T1 { int a, b; };
struct T2 { int c; double d; };
union U { T1 t1; T2 t2; };
int f() {
    U u = { { 1, 2 } }; // active member is t1
    return u.t2.c;      // OK, as if u.t1.a were nominated
}

```

— end example]

Change in 9.2 class.mem paragraph 20:

If a standard-layout class object has any non-static data members, its address is the same as the address of its first non-static data member. Otherwise, its address is the same as the address of its first base class subobject (if any). [Note: ...—end note] [**Note: The object and its first subobject are pointer-interconvertible (3.9.2, 5.2.9).** —end note]

Change in 9.5 class.union paragraph 1, splitting it into two paragraphs:

In a union, **a non-static data member is active if its name refers to an object whose lifetime has begun and has not ended ([basic.life]).** At most one of the non-static data members **of an object of union type** can be active at any time, that is, the value of at most one of the non-static data members can be stored in a union at any time. [Note: One special guarantee is made in order to simplify the use of unions: If a standard-layout union contains several standard-layout structs that share a common initial sequence (9.2), and if **a non-static data member of** an object of this standard-layout union type **contains is active and is** one of the standard-layout structs, it is permitted to inspect the common initial sequence of any of **the** standard-layout struct members; see 9.2. — end note]

The size of a union is sufficient to contain the largest of its non-static data members. Each non-static data member is allocated as if it were the sole member of a struct. [**Note: A union object and its non-static data members are pointer-interconvertible (3.9.2, 5.2.9). As a consequence,** all non-static data members of a union object have the same address. —end note]

Add a new paragraph before 9.5 (class.union) paragraph 4:

When the left operand of an assignment operator involves a member access expression (5.2.5 [expr.refl]) that nominates a union member, it may begin the lifetime of that union member, as described below. For an expression E, define the set S(E) of subexpressions of E as follows:

- If E is of the form A.B, S(E) contains the elements of S(A), and also contains A.B if B names a union member of a non-class, non-array type, or of a class type with a trivial default constructor that is not deleted, or an array of such types.
- If E is of the form A[B] and is interpreted as a built-in array subscripting operator, S(E) is S(A) if A is of array type, S(B) if B is of array type, and empty otherwise.
- Otherwise, S(E) is empty.

In an assignment expression of the form E1 = E2 that uses either the built-in assignment operator (5.18) or a trivial assignment operator (12.8), for each element x of S(E1), if modification of x would have undefined behavior under 3.8 [basic.life], an object of the type of x is implicitly created in the nominated storage; no initialization is performed and the beginning of its lifetime is sequenced after the value computation of the left and right operands and before the assignment. [Note: This ends the lifetime of the previously-active member of the union, if any (3.8 [basic.life]). — end note] [Example:

```

union A { int x; int y[4]; };
struct B { A a; };
union C { B b; int k; };
int f() {
    C c;                // does not start lifetime of any union member
    c.b.a.y[3] = 4;      // OK: S(c.b.a.y[3]) contains c.b and c.b.a.y;
                        // creates objects to hold union members c.b and c.b.a.y
    return c.b.a.y[3];   // OK: c.b.a.y refers to newly created object (see 3.8 [basic.life])
}

```

```

struct X { const int a; int b; };
union Y { X x; int k; };
void g() {
    Y y = { { 1, 2 } }; // OK, y.x is active union member (9.2)
    int n = y.x.a;
    y.k = 4;           // OK: ends lifetime of y.x, y.k is active member of union
}

```

```

__y.x.b = n; // undefined behavior: y.x.b modified outside its lifetime.
           // S(y.x.b) is empty because X's default constructor is deleted.
           // so union member y.x's lifetime does not implicitly start
}

__end_example ]

```

Library wording

Add to 18.6 (support.dynamic) paragraph 1:

```

namespace std {
    template <class T>
        constexpr T* launder(T* p) noexcept;
}

```

Add a new subclause under 18.6 (support.dynamic):

```

template <class T> constexpr T* launder(T* p) noexcept;

```

Requires: p represents the address A of a byte in memory. An object x that is within its lifetime ([basic.life]) and whose type is similar ([conv.qual]) to T is located at the address A . All bytes of storage that would be reachable through the result are reachable through p (see below).

Returns: A value of type T^* that points to x .

Remarks: An invocation of this function may be used in a core constant expression whenever the value of its argument may be used in a core constant expression. A byte of storage is reachable through a pointer value that points to an object v if it is within the storage occupied by v , an object that is pointer-interconvertible with v , or the immediately-enclosing array object if v is an array element. The program is ill-formed if T is a function type or (possibly cv-qualified) `void`.

Notes: If a new object is created in storage occupied by an existing object of the same type, a pointer to the original object can be used to refer to the new object unless the type contains `const` or reference members; in the latter cases, this function can be used to obtain a usable pointer to the new object. See [basic.life].

[**Example:**

```

struct X { const int n; };
X *p = new X{3};
const int a = p->n;
new (p) X{5}; // p does not point to new object ([basic.life])
              // because X::n is const
const int b = p->n; // undefined behavior
const int c = std::launder(p)->n; // OK

__end_example ]

```

Drafting note: see N4303 for more background on the purpose and intended usage of this function.

Change in 20.7.5 (ptr.align) paragraph 1:

Effects: If it is possible to fit `size` bytes of storage aligned by `alignment` into the buffer pointed to by `ptr` with length `space`, the function updates `ptr` to **point to represent** the first possible address of such storage and decreases `space` by the number of bytes used for alignment.

Change in 20.7.5 (ptr.align) paragraph 2:

Requires:

- `alignment` shall be a power of two
- `ptr` shall **point to represent the address of** contiguous storage of at least `space` bytes

Feature test macro

The feature test macro `__cpp_lib_launder` is proposed for the library portion of this paper.

ISO/IEC JTC1 SC22 WG21 P0137 EOM